La gestion des données

La mise à jour

Table des matières

1. Présentation des opérations de mise à jour 1
2. Mise à jour d'une catégorie 1
   2.1. Contrôle au niveau de la base de données 1
   2.2. Test fonctionnel du contrôleur http://gestion/categorie/maj/ajax/modifier.php 1
   2.3. Le contrôleur principal 2
   2.4. Le script index.html 2
   2.5. Le script index.js 2
3. Suppression d'une catégorie 4
   3.1. Contrôle au niveau de la base de données 4
   3.2. Test fonctionnel du contrôleur http://gestion/categorie/maj/ajax/supprimer.php 4
   3.3. Le script index.js 4
4. Mise à jour d'un coureur 5
   4.1. Contrôle au niveau de la base de données 5
   4.2. Test du contrôleur http://gestion/coureur/maj/ajax/modifier.php 5
   4.3. Le contrôleur principal 5
   4.1. L'interface 5
   4.2. Le script index.js 5
5. La suppression d'un coureur 6
6. Mise à jour d'une annonce 6
   6.1. Contrôle au niveau de la base de données 6
   6.2. Les contrôleurs du répertoire ajax 6
   6.3. Le contrôleur principal 6
   6.4. Le script index.js 6
   6.5. Le module majannonce 7
7. Mise à jour d'un projet et de ses compétences 8
   7.1. Le script index.js 8

Guy Verghote

Gestion des données : La mise à jour

Objectif de l'activité professionnelle

A la fin de cette activité, vous serez capable :

De mettre en place n'importe quelle interface graphique permettant la mise à jour des données d'une table ou de plusieurs tables

1. Présentation des opérations de mise à jour

La modification ou la suppression sont des opérations qui peuvent prendre plusieurs formes en fonction de la méthode choisie pour sélectionner et visualiser l'enregistrement concerné
La sélection peut se faire par une zone de liste si le volume des enregistrements est faible, par une zone de recherche avec éventuellement un système d'autocomplétion pour des tables volumineuses.
Il est dans certains cas possible de proposer une modification en mode tableau si la table comporte peu de colonnes et peu de lignes.

Nous allons mettre en évidence toutes ces possibilités à partir d'exemples concrets :

- **Sélection de l'enregistrement au moyen d'une zone de liste** : la mise à jour des catégories
- **Sélection de l'enregistrement par autocomplétion** : la mise à jour des coureurs
- **Modification en mode colonne** : pour activer/désactiver des annonces et modification des autres colonnes par un lien vers une autre page : la mise à jour des annonces
- Modifications multiples : la mise à jour des compétences associées à un projet.

2. Mise à jour d'une catégorie

2.1. Contrôle au niveau de la base de données

Toutes les contraintes check de la table categorie sont repris dans le fichier de configuration contrainte.php afin de fournir un message affichable sur l'application.
Cependant ces contrôles sont aussi définis dans la classe Métier

Le déclencheur **avantMajCategorie** contrôle le non chevauchement de l'intervalle [ageMin , ageMax] avec un intervalle existant dans le cas où l'une des colonnes est modifiée/

Le trigger interdit aussi la modification de la clé primaire

2.2. Test fonctionnel du contrôleur http://gestion/categorie/maj/ajax/modifier.php

1. Tester sous Insomnia tous les cas possibles

2.3. Le contrôleur principal

Comme pour l'ajout, il récupère les catégories pour que côté client, les contrôles sur le chevauchement et l'unicité du nom soit réalisables.

2.4. Le script index.html

Pour chaque balise Input, nous retrouvons, exprimée dans ses attributs, les règles de validité qui seront vérifiées côté client.

Champ	Nom	ageMin - ageMax
Type	text	number
required pattern maxlength minlength	true ^[A-Za-z][A-Za-z]*(?: \d{1,2})?$ 30 3	true 1 (min) 99 (max)

2.5. Le script index.js

Nous avons choisi ici de convertir les données transmises par le contrôleur en objet Map plutôt qu'en tableau.

```
const lesCategories = new Map(getData("lesCategories").map(x => [x.id, x]));
```

Map est une structure de données JavaScript qui stocke des paires clé/valeur, où la clé peut être de n'importe quel type (string, objet, etc.), contrairement à un objet classique limité aux clés string/symbol. L'ordre d'insertion est préservé, et on accède aux éléments avec get(clé), set(clé, valeur), delete(clé), has(clé) — toutes des opérations en O(1), donc bien plus rapides qu'un parcours de tableau (find/findIndex/splice qui sont en O(n)).

L'intérêt principal : quand on doit fréquemment rechercher, mettre à jour ou supprimer un élément par identifiant unique, une Map évite de parcourir toute la collection à chaque fois, rendant le code à la fois plus performant et plus lisible (lesCategories.get(id) plutôt que lesCategories.find(x => x.id === id)). Elle reste toutefois moins pratique quand on a besoin d'itérer/filtrer sur des critères autres que la clé

Lorsque l'utilisateur sélectionne une catégorie dans la zone de liste, il faut mettre à jour l'interface et conserver une trace de l'enregistrement en cours. Pour cela nous utilisons une variable globale 'categorie' qui pointe dans le tableau lesCategories l'objet sélectionné.

L'objet Map lesCategories permet de retrouver les données très facilement

```
categorie = lesCategories.get(idCategorie);
```

Puisque l'on dispose de la propriété nb (nombre de coureurs dans la catégorie) dans le tableau on peut afficher ou masquer le bouton 'btnSupprimer' en fonction de la valeur de nb

En plus de contrôle classique à l'aide de la fonction donneesValides() nous pouvons contrôler l'unicité du code, du nom, et l'absence de chevauchement.

Le contrôle d'unicité du nom consiste à vérifier si le tableau lesCatégories contient un autre enregistrement avec le même nom.
La comparaison sur le nom de devra pas tenir compte des accents ni de la casse. La fonction comparerSansAccentEtSansCasse du module util.js permet de le faire

L'utilisation d'un objet Map ne permet pas de faire cette recherche facilement il faut repasser par un tableau en utilisant l'opérateur de déstructuration

```
function nomUnique(nom) {
   return ![...lesCategories.values()].some(x => comparerSansAccentEtSansCasse(x.nom, nom) &&
x.id !== categorie.id);
}
```

Le contrôle sur les âges consiste à vérifier qu'il n'existe dans le tableau une catégorie (à l'exception de celle en cours de modification) pour laquelle il y a chevauchement. Il est plus simple d'exprimer la condition inverse et de l'inverser.
Il n'y a pas de chevauchement si l'intervalle se termine avant (max < ageMin) ou si l'intervalle commence après (min > ageMax)
Encore une fois l'objet Map doit être converti en tableau

```
function plageValide(id, min, max) {
   return ![...lesCategories.values()].some(x => x.id !== id && max >= x.ageMin && min <=
x.ageMax);
}
```

Il n'est pas difficile d'exprimer autrement la condition quand on connait la règle :
inv(A ou B) = inv(A) et inv(B)

```
return !lesCategories.some(x => x.id !== id && max >= x.ageMin && min <= x.ageMax);
```

Cela permet d'obtenir une condition plus simple à écrire mais moins évidente à trouver

Toutes ces fonctions de contrôle sont appelées depuis la fonction controler()

La fonction modifier() déclenche l'appel Ajax si une des données a été modifiée.
En cas de succès, il faut mettre à jour la variable categorie (donc le tableau), et la zone de liste

```
idR.options[idR.selectedIndex].text = categorie.nom + ' (' + categorie.id + ')';
```

La fonction appelAjax réalise l'appel du script modifier.php.
Pour une modification il faut transmettre :
- la clé primaire de l'enregistrement à modifier (primaryKey) ;
- les colonnes à modifier avec leurs nouvelles valeurs (columns).

Une fois la modification effectuée avec succès, la fonction :
- affiche le message retourné par le serveur ;
- met à jour l'objet categorie avec les nouvelles valeurs ;
- met à jour le texte de l'option correspondante dans la liste idR.

void indique que le résultat retourné par appelAjax() n'est pas utilisé. Il n'est pas indispensable

2. Réaliser les tests fonctionnels de cette interface sur la modification

3. Suppression d'une catégorie

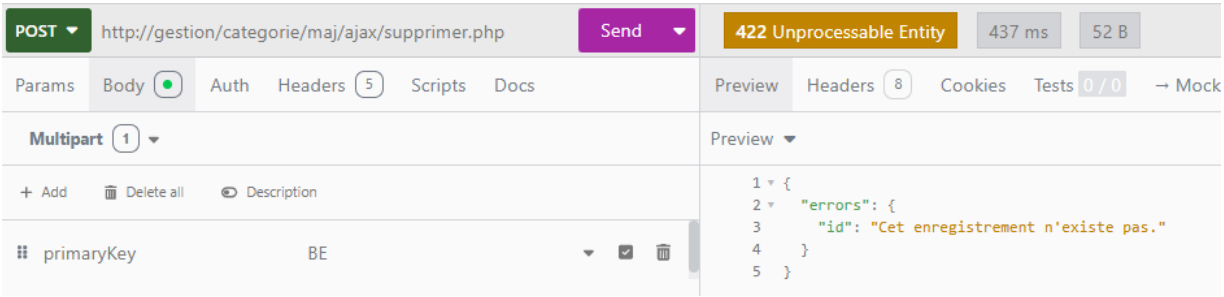
3.1. Contrôle au niveau de la base de données

La contrainte "fk_coureur_categorie" définie sur la table coureur va interdire la suppression d'une catégorie si elle comporte au moins un coureur
Par contre on peut noter qu'elle accepte qu'une catégorie change de code en appliquant la modification en cascade.
Cependant l'application ne le permet pas.

Cette contrainte est reprise dans le fichier de configuration 'contrainte.php' afin d'afficher un message significatif à l'utilisateur.
Notons encore une fois que l'application côte client ne permet pas de supprimer une catégorie possédant des coureurs (le bouton btnSupprimer n'est pas affiché)

3.2. Test fonctionnel du contrôleur http://gestion/categorie/maj/ajax/supprimer.php

3. Tester sous Insomnia tous les cas possibles



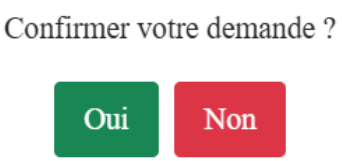
3.3. Le script index.js

La suppression est déclenchée par un clic sur le bouton 'supprimer'

```
btnSupprimer.onclick = () => confirmer(supprimer);
```

Ce bouton n'est affiché que lorsqu'aucun coureur ne fait partie de cette catégorie.

La fonction confirmer() de la bibliothèque afficher.js permet d'intégrer une demande de confirmation
La fonction supprimer() assure la suppression de la catégorie affichée en appelant via la fonction appelAjax,



En cas de succès, il faut mettre à jour l'interface et aussi mettre à jour les données en mémoire en supprimant dans le tableau lesCategories la ligne correspondante.
Au niveau de l'interface, il faut supprimer l'option actuellement affichée dans la zone de liste et appeler la fonction afficher() pour alimenter les zones de saisie de l'interface avec la nouvelle catégorie sélectionnée.

Cette fois, l'objet Map apporte un avantage au niveau de la suppression

4. Réaliser les tests fonctionnels sur la suppression

4. Mise à jour d'un coureur

4.1. Contrôle au niveau de la base de données

Toutes les contraintes définies sur la table coureur se retrouvent dans le fichier de configuration.

Au niveau du déclencheur 'avantMajCoureur', les contrôles ou actions suivantes sont réalisées

Colonne	Contrôle /action
licence	Non modifiable
nom prenom	Mise en majuscule et suppression des espaces superflus Mise en minuscule et suppression des espaces superflus
dateNaissance	Doit être compatible avec les catégories d'âge Cette règle est aussi gérée depuis la classe métier Coureur
idCategorie	Réalisation de la mise à jour si la règle précedente est vérifiée

4.2. Test du contrôleur http://gestion/coureur/maj/ajax/modifier.php

5. Tester sous Insomnia tous les cas possibles

4.3. Le contrôleur principal

Il récupère les clubs, l'intervalle des dates de naissance et les catégories afin de permettre côté client de déterminer la catégorie correspondant à la date de naissance.

4.1. L'interface

Le même système d'autocomplétions que pour la recherche sur le nom prénom est utilisé.

Au niveau de la date de naissance, l'intervalle des dates valides est dynamique. Il est calculé côté client à l'aide du tableau lesCategories

4.2. Le script index.js

Il permet de rechercher un coureur par nom/prénom grâce à une saisie avec autocomplétion, puis d'afficher ses informations (identité, sexe, date de naissance, club, licence, FFA, coordonnées, catégorie).

Il calcule automatiquement la catégorie du coureur à partir de sa date de naissance et contrôle que celle-ci se situe dans la plage de dates autorisée.

Il permet de modifier les informations après vérification des données et uniquement lorsqu'une modification a réellement été effectuée. Les changements sont envoyés au serveur via AJAX.

Il permet également de supprimer le coureur, après confirmation.

Enfin, il initialise et contrôle les champs du formulaire : noms et prénoms en majuscules, filtrage des caractères saisis, liste des clubs, dates et champs obligatoires.

6. Réaliser les tests fonctionnels de cette interface au niveau de la modification

5. La suppression d'un coureur

Aucun contrôle n'est défini dans la base de données, tout coureur pouvant être supprimé.
Coté client on retrouve le même système avec une demande de confirmation de l'opération
L'appel Ajax doit fournir uniquement la propriété primaryKey
En retour le focus est redonné au champ search afin de masquer le formulaire

7. Tester sous Insomnia le contrôleur http://gestion/coureur/maj/ajax/supprimer.php

6. Mise à jour d'une annonce

Nous appliquons ici une modification en mode colonne au niveau du champ actif.
Les enregistrements sont affichés en mode tableau.
Chaque ligne représente un enregistrement
Il est possible :

- de supprimer une annonce,
- d'activer ou de désactiver une annonce
- de rediriger le visiteur vers la page de modification d'une annonce.
- de supprimer les anciennes annonces.

6.1. Contrôle au niveau de la base de données

Le trigger 'avantModificationAnnonce' se charge de vérifier que la date reste toujours supérieure à la date du jour en cas de changement
La contrainte ck_annonce_nom_format contrôle le respect des caractères autorisés sur le nom
La contrainte ck_annonce_actif limite les valeurs autorisées pour le champ actif.
Ces contraintes sont repris dans le fichier de configuration.

6.2. Les contrôleurs du répertoire ajax

8. Tester sous Insomnia le contrôleur http://gestion/annonce/maj/ajax/deleteold.php
9. Tester sous Insomnia le contrôleur http://gestion/annonce/maj/ajax/supprimer.php
10.Tester sous Insomnia le contrôleur http://gestion/annonce/maj/ajax/modifieractif.php

6.3. Le contrôleur principal

Il charge les annonces pour les transmettre côté client.

6.4. Le script index.js

Ce script gère la liste des annonces dans l'interface d'administration. Il permet principalement de modifier leur état actif, de les supprimer individuellement ou de supprimer les anciennes annonces, avec mise à jour immédiate de l'affichage.

Les trois appels AJAX constituent le cœur des opérations avec le serveur :

Suppression d'une annonce
→ Demande au serveur de supprimer l'annonce correspondant à son identifiant. Si la suppression réussit, la ligne correspondante est retirée du tableau.

Activation / désactivation d'une annonce
→ Envoie au serveur le nouvel état de la colonne actif. En cas de succès, un message confirme l'activation ou la désactivation. En cas d'erreur, la case est remise dans son état précédent.

Suppression des anciennes annonces
→ Demande au serveur de supprimer toutes les annonces concernées. Le serveur renvoie les identifiants supprimés, puis le script retire immédiatement ces lignes du tableau et affiche un bilan du nombre d'annonces supprimées.

Les autres actions
Le script construit également dynamiquement le tableau des annonces à partir de lesAnnonces. Pour chaque annonce, il crée :

un bouton Modifier qui ouvre la page de modification ;
un bouton Supprimer avec confirmation ;
la date et le nom de l'annonce ;
une case à cocher permettant de l'activer ou la désactiver.

Pour générer une case à cocher, nous utilisons la fonction creerCheckBox() du module formulaire.js

La génération des deux boutons action est une opération que l'on peut aussi retrouver dans de nombreux scripts. Nous disposons là encore de deux fonctions dans le module formulaire.js pour automatiser cette génération

- creerBoutonModification(action)
- creerBoutonSuppression(action)

Ces deux fonctions utilisent la fonction générique creerBoutonAction(). Elle prend en paramètre un objet avec les propriétés :

- icone : caractère Unicode ou symbole SVG
- couleur : couleur du bouton
- titre : infobulle
- action : fonction exécutée au clic

11. Relancer le script insert.sql pour récupérer toutes les annonces.
12. Réaliser les tests fonctionnels sur la modification de la colonne actif et la suppression des annonces.

6.5. Le module majannonce

Il permet de modifier l'annonce dont l'identifiant est transmis en paramètre.
Afin de ne pas permettre à un hacker de deviner la valeur à transmettre pour accéder à n'importe quelle annonce, l'identifiant n'est pas de type compteur mais il est généré aléatoirement par le trigger avantAjoutAnnonce

Le script maj.js commence par configurer les éléments de l'interface, le composant tinymce, l'intervalle sur la date de l'annonce (entre un an avant et un an après la date du jour) en utilisant la fonction configurerDate du composant formulaire.js et la fonction getDateRelative du composant date.js

Puis il alimente les champs de saisie.

La modification déclenchée par le clic sur le bouton 'btnModifier' doit alimenter le champ 'description' , contrôler les données et appeler la fonction modifier

13.Tester sous Insomnia le contrôleur http://gestion/annonce/majannonce/ajax/modifier.php

14.Réaliser les tests fonctionnels de ce module.

7. Mise à jour d'un projet et de ses compétences

L'interface proposée permet :
de modifier en mode colonne le nom des projets,
d'en demander la suppression
d'ouvrir la seconde interface permettant de modifier les compétences du projet sélectionné
de façon secondaire d'ouvrir l'interface permettant l'ajout d'un projet (nom uniquement)

Ce module fonctionne de la même façon que le module annonce.
La modification en mode colonne du nom remplaçant la modification de la colonne 'actif'

7.1. Contrôle au niveau de la base de données

Le trigger avantMajCompetenceProjet interdit la modification d'un enregistrement ce qui simplifie les choses
Le trigger avantAjoutCompetenceProjet vérifie que la compétence ajoutée appartient bien au bloc 1
Le nom du projet doit être unique : uk_projet_nom
Le couple idProjet, idCompetence doit être unique uk_competenceprojet_idprojet_idcompetence

7.2. Les contrôleurs appelés en Ajax

15.Tester sous Insomnia les contrôleurs ci-dessous
http://gestion/projet/majcompetenceprojet/ajax/ajouterunecompetence.php
http://gestion/projet/majcompetenceprojet/ajax/supprimercompetence.php

7.3. Le script index.js

16.réaliser les tests fonctionnels

17.Compléter le script index.js et réaliser les tests fonctionnels